Exercise 1: Setting Up JUnit

Step 1: Create a Java Project

- Open IntelliJ IDEA or Eclipse.
- Create a new Java Project (e.g., JUnitExample).

Step 2: Add JUnit Dependency (pom.xml)

```
<dependencies>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Step 3: Create a Java Class

Calculator.java

```
public class Calculator{
    public int add(int a,int b){
        return a+b;
    }
}
```

Step 4: Create a Test Class

CalculatorTest.java

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class CalculatorTest{
    @Test
```

```
    public void testAdd(){
        Calculator calculator=new Calculator();
        assertEquals(10,calculator.add(5,5));
    }
}
```

Step 5: Run the Test

- Right-click CalculatorTest.java.
- Select Run As → JUnit Test.
- If the test passes, a green bar (✓) is displayed.

Output

JUnit version 4.13.2

Time: 0.002

OK (1 test)

Result: JUnit is successfully configured and the unit test executes successfully.

Exercise 3: Assertions in JUnit

Aim:

To use different JUnit assertions to validate test results.

AssertionsTest.java

```
import static org.junit.Assert.*;
import org.junit.Test;
public class AssertionsTest{
    @Test
```

```
    public void testAssertions(){
        assertEquals(5,2+3);
        assertTrue(5>3);
        assertFalse(5<3);
        assertNull(null);
        assertNotNull(new Object());
    }
}
```

Explanation

- assertEquals(expected,actual) → Checks whether two values are equal.
- assertTrue(condition) → Passes if the condition is true.
- assertFalse(condition) → Passes if the condition is false.
- assertNull(object) → Passes if the object is null.
- assertNotNull(object) → Passes if the object is not null.

Output

JUnit version 4.13.2

Time: 0.001

OK (1 test)

Result: All assertions pass successfully, indicating the test executed correctly.

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Aim:

To organize JUnit tests using the Arrange-Act-Assert (AAA) pattern and use @Before and @After methods.

Calculator.java

```
public class Calculator{
    public int add(int a,int b){
        return a+b;
    }
}
```

CalculatorTest.java

```
import static org.junit.Assert.*;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class CalculatorTest{
    private Calculator calculator;

    @Before
    public void setUp(){
        calculator=new Calculator();
        System.out.println("Setup Executed");
    }

    @Test
    public void testAdd(){
```

```
        // Arrange
        int a=10;
        int b=20;

        // Act
        int result=calculator.add(a,b);

        // Assert
        assertEquals(30,result);
    }

    @After
    public void tearDown(){
        calculator=null;
        System.out.println("Teardown Executed");
    }
}
```

Explanation

- Arrange: Initialize objects and test data.
- Act: Execute the method being tested.
- Assert: Verify the expected result.
- @Before: Runs before every test method (Setup).
- @After: Runs after every test method (Teardown).

Output

Setup Executed

Teardown Executed

JUnit version 4.13.2

.

Time: 0.002

OK (1 test)

Result: The test is organized using the AAA pattern, and the setup (@Before) and teardown (@After) methods execute successfully.